R3.07
SQL dans un langage de programmation

Chapitre 3
Index et optimisation



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Plan du cours

1. Comment le SGBD exécute une requête

2. Les index

3. Optimiser côté application

De la requête SQL au résultat

- SQL est **déclaratif** : on dit *quoi*, pas *comment*
- L'**optimiseur** (*planner*) choisit un **plan d'exécution** parmi de nombreux possibles :
  - parcourir toute la table (*Seq Scan*) ou passer par un index (*Index Scan*, *Bitmap Scan*) ?
  - quel algorithme de jointure (*Nested Loop*, *Hash Join*, *Merge Join*) ? dans quel ordre ?
- Il estime un **coût** pour chaque plan à partir de **statistiques** sur les données (`ANALYZE`)
- Le développeur agit sur deux leviers :
  1. la **structure physique** : index, types, contraintes
  2. la **façon d'interroger** depuis l'application : requêtes, lots, pagination, nombre d'allers-retours

Lire un plan : EXPLAIN ANALYZE

Table `Operation` : 1 million de lignes, aucun index sur `numCompte`

```
EXPLAIN ANALYZE SELECT * FROM Operation WHERE numCompte = 4242;

 Gather  (cost=1000.00..13561.93 rows=6 width=25) (actual time=5.211..18.546 rows=2 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on operation  (cost=0.00..12561.33 rows=2 width=25)
                                       (actual time=10.874..14.583 rows=1 loops=3)
         Filter: (numcompte = 4242)
         Rows Removed by Filter: 333333
 Planning Time: 0.284 ms
 Execution Time: 18.609 ms
```

- `EXPLAIN` : plan **estimé** ; `EXPLAIN ANALYZE` : exécute réellement et mesure
- `cost` : unité arbitraire ; `rows` estimées vs réelles ; `Rows Removed by Filter` : travail gaspillé
- Ici : 1 million de lignes lues pour en garder 2

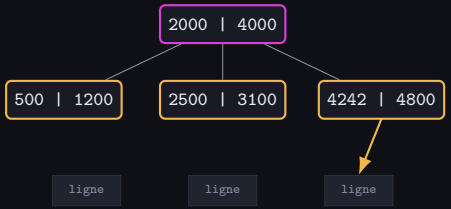
Plan du cours

1. Comment le SGBD exécute une requête

2. Les index

3. Optimiser côté application

Un index B-tree



- Arbre **trié et équilibré** sur la valeur de la colonne
- Recherche en $O(\log n)$ au lieu de $O(n)$
- Sert pour =, <, >, `BETWEEN`, `ORDER BY`, jointures
- Type par défaut de `CREATE INDEX`

Créer un index et mesurer

```
CREATE INDEX idx_operation_compte ON Operation(numCompte);
```

```
Bitmap Heap Scan on operation  (cost=4.47..28.03 rows=6 width=25) (actual time=0.017..0.019 rows=2
    loops=1)
  Recheck Cond: (numcompte = 4242)
  ->  Bitmap Index Scan on idx_operation_compte  (cost=0.00..4.47 rows=6 width=0)
                                                 (actual time=0.013..0.013 rows=2 loops=1)
        Index Cond: (numcompte = 4242)
Execution Time: 0.029 ms
```

Requête (mesures PostgreSQL 16, 1 M d'opérations)	sans index	avec index
Opérations d'un compte	19 ms	0,03 ms
Somme des opérations des comptes d'un client	39 ms	0,06 ms
`DELETE` de 50 comptes (vérif. de clé étrangère)	2 182 ms	0,6 ms

Quels index existent déjà ?

- PostgreSQL crée **automatiquement** un index pour chaque `PRIMARY KEY` et chaque contrainte `UNIQUE`
- Il ne crée pas d'index sur les **clés étrangères** :
  - jointures et recherches « les opérations du compte X » : parcours complet
  - `DELETE`/`UPDATE` sur la table référencée : pour chaque ligne supprimée, la base vérifie la table fille… en la parcourant entièrement

⇒ Règle pratique : **indexer les colonnes de clé étrangère** utilisées dans les jointures

- Lister les index : `\di` dans `psql`, ou la vue `pg_indexes`

Quand l'index n'est pas utilisé

Fonction appliquée à la colonne

```
-- index sur nomClient ignore :
WHERE lower(nomClient) = 'dupont'
-- solution : index sur l'expression
CREATE INDEX ON Client(lower(nomClient));
```

LIKE par préfixe

```
-- ignore avec une collation fr/en :
WHERE nomClient LIKE 'Dup%'
CREATE INDEX ON Client
  (nomClient varchar_pattern_ops);
-- '%pont' : aucun B-tree ne peut aider
```

Index composite : l'ordre compte

```
CREATE INDEX ON Operation
  (numCompte, dateOperation);
-- utilise :
WHERE numCompte = 3 AND dateOperation > ?
WHERE numCompte = 3
-- PAS utilise (2e colonne seule) :
WHERE dateOperation = ?
```

Faible sélectivité

- `typeOperation = 'DEPOT'` : 1 ligne sur 3
- l'index aide peu (54 ms $\rightarrow$ 28 ms) ; l'optimiseur préfère parfois le parcours complet

Un index a un coût

- Chaque `INSERT`, `UPDATE`, `DELETE` doit aussi **mettre à jour tous les index** de la table
- Mesure : insertion de 300 000 opérations
  - sans index secondaire : **0,13 s**
  - avec 4 index secondaires : **1,3 s** (10 fois plus lent)
- Les index occupent de l'espace disque et de la mémoire

Démarche

1. identifier les requêtes **fréquentes** ou **lentes** de l'application
2. les analyser avec `EXPLAIN ANALYZE`
3. créer l'index qui correspond **exactement** à la clause `WHERE`/`JOIN`/`ORDER BY`
4. re-mesurer ; supprimer les index inutiles

Plan du cours

1. Comment le SGBD exécute une requête

2. Les index

3. Optimiser côté application

Réduire les allers-retours : les lots (*batch*)

```
c.setAutoCommit(false);                          // une seule transaction
try (PreparedStatement ps = c.prepareStatement(
      "INSERT INTO Operation VALUES (?, ?, ?, ?, ?)")) {
  for (Operation op : operations) {
    ps.setInt(1, op.getNum()); /* ... */
    ps.addBatch();                               // empile au lieu d'envoyer
    if (++n % 1000 == 0) ps.executeBatch();      // envoie par paquets de 1000
  }
  ps.executeBatch();
}
c.commit();
```

- Une requête = un aller-retour réseau + (en autocommit) une écriture disque synchrone
- Avec le pilote PostgreSQL, ajouter `?reWriteBatchedInserts=true` à l'URL JDBC regroupe les lignes en `INSERT` multi-valeurs
- Gain typique : un **ordre de grandeur** ou plus (à mesurer en TP)

Pagination : `OFFSET` vs pagination par clé

```sql
-- page 45 001 : lit et jette 900 000
    lignes
SELECT * FROM Operation
 ORDER BY numOperation
 LIMIT 20 OFFSET 900000;
```

79 ms

```sql
-- on part de la derniere cle affichee
SELECT * FROM Operation
 WHERE numOperation > 900000
 ORDER BY numOperation LIMIT 20;
```

0,02 ms

- `OFFSET` est simple mais son coût croît avec le numéro de page
- La pagination par clé (*keyset*) utilise l'index et reste constante

Autres règles d'hygiène

- `PreparedStatement` partout : sécurité (injection SQL) et réutilisation du plan par le serveur
- Sélectionner **les colonnes utiles** plutôt que `SELECT *`
- Filtrer, trier, agréger **en SQL** plutôt qu'en Java (ne pas rapatrier 1 M de lignes pour en compter 10)
- Éviter les requêtes dans une boucle : le problème « **N+1** » (on le retrouvera avec JPA : TP4 et chapitre 6)
- Réutiliser les connexions : **pool** (HikariCP)
- Mesurer avant d'optimiser : `EXPLAIN ANALYZE`, `System.nanoTime()`, `pg_stat_statements`

Questions ?



Chouki.Tibermacine@univ-ubs.fr